

周报 —— 评测体系提速、评分基准修复与模型换代决策

作者:Xiangzhu Yan

周期:2026-07-31 ~ 08-01

一句话总结

本周把"给 AI 诊断系统打分"这件事从 **80 分钟压缩到 10 分钟**(快 7.5 倍),顺手挖出并修复了一个让历史分数虚假下跌的评分错误;然后利用变快的评测,在两天内完成了一次**模型换代选型**和一次"**最贵的模型到底值不值**"的对照实验,两个实验的总花费约 10 美元。结论:我们本地免费部署的 27B 小模型,和参数量大 100 倍、按量收费的顶级模型,在我们的任务上得分几乎一样。

背景:为什么要做这件事

我们有一套"考卷"来衡量 AI 诊断系统的质量:30 道由专家审定的维修问答题(golden 测试集),每次改动系统后重考一遍,看分数有没有退步。

问题是**考一次要 55~80 分钟**。上周做手册索引项目(HARNESS-30)时,一次 A/B 对比要烧掉 3.5 小时,每次"改一行代码 → 看分数"的循环要等一小时 —— 评测速度成了整个研发节奏的瓶颈。本周目标(issue #225):**20 分钟以内**。

成果一:评测提速 80.5 → 10.7 分钟(超额完成目标)

分三步实现,每一步都用"分数不变"作为硬验收标准:

第 1 步:让"判卷"和"答题"同时进行(80.5 → 54.6 分钟)

原来的流程是串行的:第 1 题答完 → 判卷 → 第 2 题答题 → 判卷.....

但判卷用的是云端服务(不占本地 GPU),完全可以在第 2 题答题的**同时**给第 1 题判卷。重构后判卷时间几乎完全消失在答题时间的影子里。

验收:同一天同内容对照,分数 0.790 vs 0.791,一致。

第 2 步:两张显卡各跑一份模型,同时答两道题(54.6 → 26.6 分钟)

服务器有两张显卡。原计划让一份模型同时处理多道题(批处理),但发现我们用的推理软件(Ollama)对这个模型**不支持批处理** —— 于是改为每张显卡完整部署一份模型,两道题真正并行,互不拖慢。

验收:分数 0.790,与串行分毫不差,每道题的速度也不受影响。

第 3 步:换更好的推理引擎 vLLM(26.6 → 10.7 分钟,随模型换代实现)

vLLM 是工业界标准的推理引擎,原生支持批处理(6 道题同时答)、能同时吃满两张显卡。配合新模型(见成果三),全量考试压到 10.7 分钟。

****当前状态:考一次 = 10.7 分钟,分数与历史完全可比,这个速度已接近**

物理下限(最长的一道题本身就要跑 4 分钟,再快就得改产品本身了)。**

成果二:挖出一个让全系统"虚假掉分"的评分错误(#234)

发现过程

给提速改动做验收时,分数从基准的 0.891 掉到 0.790。按流程做了同日对照实验:**旧代码在今天跑也是 0.790** —— 说明掉分和提速改动无关,是系统本身在两天前就"退步"了。

排查(三步定位,全程不需要占用 GPU)

1. **排除判卷老师的嫌疑**:把两天前的高分答卷原封不动拿给今天的判卷系统重判 → 分数复现(0.891 → 0.898)。判卷没变。

2. **解剖分数构成**:掉分全部集中在"引用的章节编号是否与标准答案一致"这类**对号入座**的维度;而"答案内容质量"反而**上升**了。

说明:答案没变差,是"标准答案的编号格式"对不上了。

3. **锁定根因**:上周手册索引项目切换了章节编号体系(新格式)。当时做 A/B 测试用的"新格式标准答案文件"只是临时放在服务器上,**从未写进代码**;后来一次常规代码同步把它冲掉了 —— 从那以后,系统用新格式引用章节,考卷却按旧格式对答案,张冠李戴,全线虚假扣分。

修复与教训

修复:标准答案文件的选择逻辑写进代码,跟随索引开关自动切换。修复后全量重考:**0.889,与历史基准完全吻合** —— 系统从来没有退步过。

****教训(已写入团队备忘)**:任何影响评分的配置必须进代码仓库,不能靠服务器上的手工文件状态 —— 那种状态会在下一次代码同步时无声消失。 **

成果三:模型换代选型(烤箱赛)—— 新模型转正

现役模型 qwen3.5 已服役较久,本周用变快的评测体系做了一次严格选型:

代码完全冻结,只换模型,同一套考卷考两轮取平均。

候选	两轮平均分	不及格题数	考一轮耗时	结论
现役 qwen3.5-27B(参照)	0.889	0	51 分钟	—
新 Qwen3.6-27B	0.881(持平)	0	10.7 分钟	☒ 转正
新 Qwen3.6-35B(MoE 轻量版)	0.815	2~3 题,稳定复现	6.5 分钟	☐ 淘汰

两个值得汇报的细节:

• **35B 落选的原因很有教育意义**:它名义参数量更大(35B),但采用了"每次只动用 3B 参数"的省算力架构(MoE),等效能力只有约 10B 级。它挂掉的题不是随机失误,而是**每轮都在同样的地方以同样的方式失败**(该坚持查手册时过早放弃、输出格式失守)。 ****结论**:按名义参数量选型是陷阱,必须实测。 **

• 选型过程还逼出并修复了两个潜伏的代码缺陷(空工具列表导致新引擎报错、一个从未被触发过的配置崩溃),这是"换个环境跑一遍"的附带价值。正式切换生产环境的工作已立项(#237),待排期。

成果四:"最贵的模型值不值"对照实验(总花费约 10 美元)

一个自然的疑问:我们用的是免费的本地 27B 小模型,如果换成市面上最强的按量收费模型,分数会不会质变?本周直接测了:

模型	参数量	平均分	考一轮耗时	考一轮费用
本地 Qwen3.6-27B	27B	0.881	10.7 分钟	0 元(实验室服务器)
GLM-5.2(智谱旗舰)	744B	0.876	3.9 分钟	~\$3-6
Kimi K3(月之暗面旗舰)	2.8T	0.888	6.0 分钟	~\$6-8

参数量跨越两个数量级,分数在同一条误差带内 —— 没有质变。

更有说服力的是细节:那几道所有模型都考不好的题,顶级模型和小模型

错得分毫不差(有一道题三个模型都得 0.578)—— 说明瓶颈在考题的标准答案口径和评分尺度上,不在模型能力上。唯一随模型规模稳定上升的是"该拒答时拒答"的判断力(+0.075),幅度有限。

这个实验对架构的意义(已写入设计文档 §8.4)

- 1. 系统能力主要来自我们自建的"脚手架"(手册索引、工具纪律、提示工程),不来自模型参数 —— 每一分投在脚手架上的钱都在免费升级所有模型;
- 2. 窄任务子模块用"最便宜的够用模型",该结论与行业最新实践一致(业界实测异构配模可省 40-60% 成本;我们因为子模块完全本地化,节约比例更高);
- 3. 贵的模型只花在刀刃上:调度决策、拒答判断、未知任务探路;
- 4. 今后每个新模块上线前都跑一轮这种选型(现在只要 4-10 分钟/模型)。

负结果(也是结果)

- MTP 投机解码(一种理论上无损的推理加速技术):实测在我们的负载上更慢(11.1 vs 10.7 分钟)且分数有可疑波动,已回滚并记档。原因:该技术加速的是"逐字生成"阶段,而我们的负载瓶颈在"批量读入手册内容"阶段 —— 加速了不痛的地方。
- 15 路高并发:并行度不是越大越好,实测 6-8 路是这套硬件的最优点,15 路反而更慢并引发超时。已把 6 路定为标准配置。

数字汇总

- 评测耗时:80.5 → 54.6 → 26.6 → 10.7 分钟(累计 7.5 倍)
- 分数基准:0.889(修复 #234 后重新封存,四方模型交叉验证过)
- 本周全部云端实验费用:约 \$10.4(含约 250 次判卷调用)
- 代码变更:5 个 PR 全部合并(#233 提速、#235 评分修复、#236 选型、#238 天花板实验),1 个新 issue 待排期(#237 生产切换)
- 附带修复的潜伏缺陷:3 个

下周建议(待讨论)

1. 执行 #237:新模型 + vLLM 的生产环境正式切换与端到端验证
2. 修"慢性低分题":四个模型共同证明那几题的丢分是标准答案口径问题,
修口径 = 涨分 + 提速 + 降低用户等待,一举三得
3. (远期)异构全流程实验:待端到端测试集就绪后,量化"调度用大模型 +
执行用小模型"在完整业务流上的真实节约

原始数据与详细分析见 docs/reports/2026-08-01-eval-speedup-and-model-bakeoff.md 与 docs/reports/2026-08-01-sota-ceiling-experiment.md。